

Linux Cryptography Lab

Breaking the Caesar Cipher & Decrypting Files

Domain	Tools Used	Difficulty	Status
Cryptography / Linux	Bash, OpenSSL, tr	Beginner	Completed

Project Overview

This lab activity is part of the Google Cybersecurity Certificate curriculum. As a security analyst, understanding encryption and decryption is critical to protecting data and responding to incidents. In this hands-on lab, I used Linux command-line tools to break a Caesar cipher and decrypt an encrypted file, simulating a real-world data recovery scenario.

Learning Objectives

- Explore the Linux home directory and read file contents using command-line tools
- Locate hidden files using the `ls -a` flag
- Understand and apply Caesar cipher decryption using the `tr` command
- Decrypt an AES-256-CBC encrypted file using OpenSSL
- Understand how symmetric encryption and key-based decryption work

Background: Caesar Cipher

The Caesar cipher is one of the oldest encryption techniques, named after Julius Caesar who used it in his private correspondence. It works by shifting each letter of the alphabet by a fixed number of positions. For example, with a shift of 3 to the left, the letter 'D' maps to 'A' and 'E' maps to 'B'. While far too simple for modern use, it provides an excellent foundation for understanding substitution ciphers and the broader concept of cryptographic algorithms.

Task 1: Read the Contents of a File

The lab begins in the home directory `/home/analyst`. The first step was to explore its contents and read an important README file.

Step 1 — List the home directory

Used the `ls` command to view files and subdirectories:

```
ls /home/analyst
```

Output:

```
Q1.encrypted  README.txt  caesar
```

Three items were found: two files (`Q1.encrypted` and `README.txt`) and a subdirectory (`caesar`).

Step 2 — Read the README file

Used `cat` to display the file contents:

```
cat README.txt
```

Output:

```
Hello,  
All of your data has been encrypted. To recover your data, you will need to  
solve a cipher. To
```

The message advised that the `caesar` subdirectory contains a hidden file with further instructions.

Task 2: Find a Hidden File & Decrypt the Caesar Cipher

Step 1 — Navigate to the caesar directory

```
cd caesar
```

Step 2 — List all files including hidden ones

In Linux, hidden files start with a period (.). The `-a` flag reveals them:

```
ls -a
```

Output:

```
.  ..  .leftshift3
```

The hidden file `.leftshift3` was discovered. Its name is a hint: the Caesar cipher shift is 3 positions to the left.

Step 3 — Read the encrypted file

```
cat .leftshift3
```

The output appeared scrambled — this is the Caesar-ciphered text.

Step 4 — Decrypt using the tr command

The `tr` (translate) command was used to shift all alphabetic characters 3 places back to their original positions:

```
cat .leftshift3 | tr "d-za-cD-ZA-C" "a-zA-Z"
```

How this works:

- `"d-za-cD-ZA-C"` — The encrypted alphabet — letters shifted 3 to the right (the cipher state)
- `"a-zA-Z"` — The original alphabet — maps back each encrypted letter to its plaintext letter
- `Pipe ( | )` — Sends the output of `cat` directly as input to the `tr` command

Output (decrypted message):

In order to recover your files you will need to enter the following command:

```
openssl aes-256-cbc -pbkdf2 -a -d -in Q1.encrypted -out Q1.recovered -k ettubrute
```

The decrypted message revealed the exact OpenSSL command needed for the next task. Note: there was no need to copy it manually — the command was provided directly in Task 3.

Step 5 — Return to the home directory

```
cd ~
```

Task 3: Decrypt the Encrypted File

With the decryption command now known, the final task was to decrypt the file `Q1.encrypted` to recover the hidden message.

Step 1 — Run the OpenSSL decryption command

```
openssl aes-256-cbc -pbkdf2 -a -d -in Q1.encrypted -out Q1.recovered -k ettubrute
```

Command breakdown:

Flag / Component	Description
<code>openssl</code>	Command-line tool for cryptographic operations
<code>aes-256-cbc</code>	Cipher algorithm: AES (Advanced Encryption Standard), 256-bit key, CBC mode
<code>-pbkdf2</code>	Adds extra security by applying key derivation function to the password
<code>-a</code>	Specifies base64 encoding (the file is stored in base64 format)
<code>-d</code>	Decrypt mode (as opposed to <code>-e</code> for encrypt)
<code>-in Q1.encrypted</code>	Specifies the input (encrypted) file

<code>-out Q1.recovered</code>	Specifies the output (decrypted) file name
<code>-k ettubrute</code>	The decryption password/key: ettubrute

Step 2 — Verify the decrypted file was created

```
ls
```

The file `Q1.recovered` now appeared in the directory listing, confirming the decryption was successful.

Step 3 — Read the recovered message

```
cat Q1.recovered
```

The decrypted file revealed the hidden message, completing the lab.

Commands Summary

Command	Purpose	Example
<code>ls</code>	List directory contents	<code>ls /home/analyst</code>
<code>ls -a</code>	List all files including hidden	<code>ls -a</code>
<code>cat</code>	Display file contents	<code>cat README.txt</code>
<code>cd</code>	Change directory	<code>cd caesar</code>
<code>cd ~</code>	Return to home directory	<code>cd ~</code>
<code>tr</code>	Translate/substitute characters	<code>tr "d-za-cD-ZA-C" "a-zA-Z"</code>
<code>openssl</code>	Encrypt/decrypt files	<code>openssl aes-256-cbc ...</code>
<code>pipe ()</code>	Send output to another command	<code>cat file tr ...</code>

Key Takeaways

- Hidden files in Linux begin with a period (.) and are invisible to a standard `ls` command — the `-a` flag is required to reveal them.
- The Caesar cipher is a substitution cipher that can be broken with the `tr` command by mapping the shifted character set back to the original alphabet.
- AES-256-CBC is a modern, industry-standard symmetric encryption algorithm. OpenSSL provides command-line access to encrypt and decrypt files using this cipher.

- The -pbkdf2 flag in OpenSSL applies Password-Based Key Derivation Function 2, which strengthens the key derived from the password and makes brute-force attacks harder.
- The pipe (|) operator is a powerful Linux tool that chains commands together, passing the output of one directly as input to the next.
- Understanding both classical ciphers (Caesar) and modern encryption (AES-256) is essential for a well-rounded foundation in cryptography and cybersecurity.

Skills Demonstrated

Technical Skills	Conceptual Knowledge
• Linux command-line navigation • File reading and exploration • Hidden file discovery (ls -a) • Character translation with tr • File decryption with OpenSSL • Command piping	• Caesar cipher mechanics • Symmetric encryption principles • AES-256-CBC algorithm • Key derivation (PBKDF2) • Base64 encoding in cryptography • Cryptographic attack awareness

Conclusion

This lab provided practical, hands-on experience with real cryptographic tools used in the cybersecurity field. By working through a simulated ransomware-style scenario — where all files were "encrypted" and a cipher needed to be solved to recover them — I developed both the technical skills and the analytical mindset required of a security analyst. The exercise reinforced the importance of understanding how encryption works at a fundamental level, which is essential for defending systems, responding to incidents, and advising organisations on security controls.